

Pre-Course Primer: LLM Internals & Performance Engineering

This document provides foundational knowledge, architectural reasoning, and critical trade-offs aligned with your upcoming Module 2 syllabus. It distinguishes established mathematical facts from empirical architectural choices.

1. Linear Classification & Optimization (Prep for Mar 18 & 25)

Before understanding a 100-billion parameter model, you must understand the optimization of a single matrix.

Linear Classification Models

- **Mechanism:** A classifier maps an input vector x to output logits y via a weight matrix W and bias b : $y = xW + b$.
- **The LLM Head:** The final layer of an LLM (the "head") is essentially a massive linear classifier. It takes the final hidden state of the transformer and projects it across the entire vocabulary space to output logits. These logits are passed through a Softmax function to yield a probability distribution over the vocabulary.
- **Fact:** Softmax normalizes logits into probabilities, but it is highly sensitive to large values (exponential scaling).

Gradient Descent & SGD Modifications

- **Stochastic Gradient Descent (SGD):** Instead of calculating the gradient over the entire dataset (which is computationally impossible for LLMs), SGD calculates the gradient on a small "mini-batch."
- **The Problem with Vanilla SGD:** It struggles with "ravines" in the loss landscape, oscillating across slopes rather than moving directly to the local minimum.
- **Adam (Adaptive Moment Estimation):** The industry standard for training LLMs.
 - *Mechanism:* It maintains two moving averages for each parameter: the first moment (mean of gradients, like momentum) and the second moment (uncentered variance of gradients).
 - *Architectural Reasoning:* By dividing the learning rate by the square root of the second moment, Adam effectively gives infrequent parameters larger updates and frequent parameters smaller updates. This is critical for NLP, where word frequencies follow a long-tail distribution (Zipf's Law).

2. Deep Learning Primitives & Sequences (Prep for Apr 1 & 8)

Fully Connected Neural Networks (FCNNs/MLPs)

- **Mechanism:** FCNNs introduce non-linearity. A linear transformation followed by a non-linear activation function (like ReLU or GeLU).
- **Fact (Universal Approximation Theorem):** A feed-forward network with a single hidden layer containing a finite number of neurons can approximate continuous functions on compact subsets of $\mathbb{R}^n$, given a suitable activation function.
- **Role in LLMs:** FCNNs act as the "memory" or "knowledge retrieval" blocks within transformer layers, processing the context aggregated by the attention mechanism.

Tokenization

- **Mechanism:** LLMs do not read text; they read arrays of integers. Tokenization maps sub-words to these integers (e.g., Byte Pair Encoding (BPE)).
- **Trade-off: Vocabulary Size vs. Sequence Length.**
 - A smaller vocabulary (e.g., character-level) means longer sequences, straining the context window and compute.
 - A larger vocabulary means shorter sequences, but drastically inflates the size of the embedding matrix and the final output head, requiring more memory.

The Problem with RNNs

- **Mechanism:** Recurrent Neural Networks (RNNs) process tokens sequentially, maintaining a "hidden state" that updates with each new token.
- **Architectural Flaw:** Sequential processing prevents parallelization across GPUs. Furthermore, through backpropagation through time (BPTT), gradients tend to vanish or explode, making it mathematically difficult for the network to retain information from token 1 by the time it reaches token 1000.

3. The Transformer Architecture (Prep for Apr 15 & 22)

The transformer solved the RNN's sequential bottleneck by processing all tokens simultaneously using Attention.

Scaled Dot-Product Attention

This is the core mathematical engine of modern AI.

$$\text{Attention}(Q, K, V) = \text{softmax} \left(\frac{QK^T}{\sqrt{d_k}} \right) V$$

-

Internal Mechanism:

1. **Q (Query), K (Key), V (Value):** For every token, linear projections create a Query vector (what I am looking for), a Key vector (what I contain), and a Value vector (my actual content).
2. QK^T **(Dot Product):** Taking the dot product of every Query with every Key computes a similarity score. High dot product = high relevance.
3. **Scaling ($\sqrt{d_k}$):** *Architectural reasoning:* As the dimension d_k grows, the dot products grow large in magnitude. This pushes the softmax function into regions with extremely small gradients (saturation). Dividing by $\sqrt{d_k}$ stabilizes the gradients.
4. **Multiplication with V:** The softmax scores are used as weights to sum up the Value vectors. The output for a token is a weighted average of all other relevant tokens' values.

Decoder-Only Architecture (The Standard LLM)

- Most modern LLMs (GPT, Llama, Mistral) drop the encoder from the original Transformer. They are "autoregressive decoders."
- **Masked Self-Attention:** They use a lower-triangular mask on the QK^T matrix to ensure a token can only attend to previous tokens, never future tokens (which would be cheating during training).

4. Modern Architecture & Performance Engineering (Prep for Apr 29 & May 6)

This is where the theoretical meets the physical constraints of GPU memory and bandwidth.

Modern Components (Working Theories & Empirical Standards)

- **RoPE (Rotary Position Embeddings):** Instead of adding absolute position vectors (like the original transformer), RoPE encodes relative position by rotating the Query and Key representations in the complex plane. *Benefit:* Better extrapolation to sequence lengths longer than seen in training.
- **SwiGLU Activation:** Replaces standard ReLU in the FCNN layers. It is empirically proven to yield better performance per parameter, though the strict mathematical reason *why* it outperforms others is still a working theory in the literature.

Practical LLM Inference: The Memory Wall

LLM inference is generally **memory bandwidth bound**, not compute bound. Generating a token requires loading the entire model's weights from HBM (High Bandwidth Memory) to the compute cores.

- **The KV Cache:** To avoid recomputing the Keys and Values for previous tokens during generation, they are cached in GPU memory.
- **GQA (Grouped Query Attention):** *Trade-off resolution.* Multi-Head Attention (MHA) creates unique K and V heads for every Q head, leading to massive KV caches that exhaust VRAM. Multi-Query Attention (MQA) uses 1 K and V head for all Q heads (degrades quality). GQA groups multiple Q heads to share a single K/V head, striking the optimal balance between inference memory reduction and model capability.

Fine-Tuning and LoRA (Low-Rank Adaptation)

- **The Problem:** Full fine-tuning of a 70B parameter model requires updating 70B weights, necessitating massive clusters of GPUs simply to hold the optimizer states (e.g., Adam's momentum variables take 2-3x the memory of the model weights themselves).
- **LoRA Mechanism:** LoRA freezes the pre-trained weight matrix W_0 . Instead of updating W_0 , it learns a smaller weight update ΔW .
- **The Math:** ΔW is constrained by low-rank decomposition. It is represented as the product of two smaller matrices, B and A .

$$W_{\text{new}} = W_0 + BA$$

If W_0 is $10,000 \times 10,000$ (100M parameters), and rank $r = 8$, B is $10,000 \times 8$ and A is $8 \times 10,000$ (160k parameters total).

- **Performance Consideration:** LoRA reduces trainable parameters by up to 10,000x, drastically reducing VRAM requirements and allowing fine-tuning on consumer-grade GPUs, with the trade-off being a highly constrained update space that cannot learn entirely new foundational concepts, but is excellent for style transfer or specific format instruction.

5. The Current Open/API Model Landscape

Understanding the current ecosystem requires distinguishing between underlying architectures (Dense vs. Mixture-of-Experts) and the intended computational environment (Edge vs. Data Center).

DeepSeek (China)

Models: DeepSeek-V3.2, DeepSeek-R1-0528, DeepSeek-V3-0324

- **Introduction:** DeepSeek leads the frontier of hyper-efficient AI. Their models utilize DeepSeek Sparse Attention (DSA) and advanced MoE (Mixture of Experts) architectures.
- **Strengths:** Market-disrupting efficiency. DeepSeek-V3.2 currently matches or beats proprietary models (like GPT-5.2 and Claude Opus) on production coding and math benchmarks at a fraction of the inference cost. R1-series models excel in verifiable Chain-of-Thought (CoT) reasoning.
- **Weaknesses:** While highly logical, the "reasoner" variants can struggle with context comprehension and consistency in creative writing or roleplay scenarios. There are also reported alignment and censorship constraints based on their origin.

Qwen / Alibaba Cloud (China)

Models: Qwen3 series (Coder, Thinking, VL), Qwen2.5 series

- **Introduction:** Alibaba's flagship open-weight series. The Qwen3 family utilizes an MoE architecture spanning from 0.6B to massive 480B parameter configurations, featuring specialized "Thinking" modes for long-horizon planning.
- **Strengths:** Exceptional cost-to-performance ratio and native multilingual capabilities (119 languages). Qwen3 MoE configurations maintain very low active parameters per token, making high-end coding and reasoning accessible on moderate hardware.
- **Weaknesses:** The most glaring empirical weakness is a deficit in general "world knowledge" (pop culture, media, niche history), which leads to severe hallucinations even at low temperatures. It also exhibits high baseline alignment/caution which can hinder complex, unconstrained problem-solving.

OpenAI (USA)

Models: gpt-oss-120b, gpt-oss-20b

- **Introduction:** OpenAI's pivot into the open-weights ecosystem. These are MoE models trained with reinforcement learning techniques derived from their internal o-series (o3/o4).
- **Strengths:** State-of-the-art agentic tool use (web browsing, function calling) and reasoning. They are natively quantized in MXFP4, allowing the 120B model (117B total, 5.1B active parameters) to run efficiently on a single 80GB GPU.
- **Weaknesses:** While the weights are open, the training data and precise pre-training methodologies remain a proprietary black box. Furthermore, "raw" output formatting requires explicit handling of harmony schemas to function correctly.

Moonshot AI (China)

Models: Kimi-K2.5, Kimi-K2-Instruct, Kimi-K2-Thinking

- **Introduction:** Moonshot's native multimodal architecture is built around ultra-long context limits and subagent orchestration (swarms).

- **Strengths:** A massive 256K native context window handled with high retrieval accuracy. It dominates in complex document extraction, visual OCR tasks, and executing multi-step agent workflows.
- **Weaknesses:** Extreme verbosity. The model tends to generate significantly more output tokens than competitors (over-engineering code or explanations), which heavily inflates the effective API cost. Additionally, in highly complex agent swarms, subagents can suffer from conceptual drift.

Prime Intellect (USA)

Models: INTELLECT-3

- **Introduction:** A 106B-parameter MoE model built by fine-tuning Zhipu's GLM-4.5-Air base model using massive asynchronous Reinforcement Learning (RL) via their custom "prime-rl" stack.
- **Strengths:** A powerful demonstration of decentralized/open reinforcement learning. It performs exceptionally well on math and science benchmarks for its active weight class (12B active parameters).
- **Weaknesses:** Dependent on the quality of open RL environments; it occasionally trails absolute frontier models (DeepSeek V3.2) in highly constrained, production-level coding workflows.

Zhipu AI / Z.ai (China)

Models: GLM-5, GLM-4.7, GLM-4.5

- **Introduction:** Flagship bilingual foundation models with strong long-context and agent-oriented capabilities.
- **Strengths:** Very intelligent bare-prompt reasoning. It can solve complex logic problems from scratch without relying heavily on strict prompting frameworks.
- **Weaknesses:** In fully automated tasks (where the model orchestrates itself without user intervention), it can derail or hallucinate tool calls over long horizons. Official API endpoints have historically faced speed constraints compared to DeepSeek or Qwen.

Meta (USA)

Models: Llama-3.3-70B-Instruct, Meta-Llama-3.1-8B, Meta-Llama-Guard-3

- **Introduction:** The anchor of the Western open-source ecosystem, relying on traditional dense transformer architectures rather than MoE.
- **Strengths:** Extremely robust, predictable performance across chat, coding, and general knowledge. Massive community support guarantees immediate integration into fine-tuning frameworks like LoRA and execution engines like vLLM.
- **Weaknesses:** Because they are dense models, inference requires calculating *every* parameter for *every* token. A dense 70B model requires significantly more compute

throughput than an MoE model with 70B total but only 10B active parameters.

Google (USA)

Models: Gemma-3-27b-it, Gemma-2-2b-it, Gemma-2-9b-it

- **Introduction:** Google's lightweight, open-weight models based on the architecture of their flagship Gemini models.
- **Strengths:** Incredible instruction following and prose generation at small parameter counts. Highly optimized for on-device/edge inference.
- **Weaknesses:** Empirically, they struggle with deep mathematical and logical reasoning compared to MoE counterparts like Qwen or DeepSeek at similar parameter scales.

NVIDIA (USA)

Models: Nemotron-3-Super-120b, Nemotron-Nano-V2-12b, Llama-3_1-Nemotron-Ultra

- **Introduction:** NVIDIA's enterprise-grade models, frequently fine-tuned variants of existing architectures or highly optimized MoEs built for their proprietary hardware.
- **Strengths:** Unparalleled stability, robust safety guardrails, and highly deterministic behavior for enterprise data pipelines and RAG (Retrieval-Augmented Generation).
- **Weaknesses:** Heavily aligned, meaning they may refuse benign requests if they trigger safety heuristics, and they generally lack the creative versatility seen in Hermes or standard Llama iterations.

NousResearch (USA)

Models: Hermes-4-405B, Hermes-4-70B

- **Introduction:** Community-driven models heavily fine-tuned on verified Chain-of-Thought traces.
- **Strengths:** Exceptionally uncensored and steerable. They will execute exactly what is prompted, making them ideal for highly customized programmatic workflows where alignment artifacts cause systemic failures.
- **Weaknesses:** Performance is bounded by the base model (typically Llama), and the 405B variant requires institutional-grade hardware (multiple 80GB GPUs) simply to load into memory.

6. Decoding Model Nomenclature & Tags

Model names in provider platforms typically follow a strict taxonomy indicating their architecture, training stage, and specialized modality. Understanding these tags is crucial for calculating VRAM limits and deployment costs.

Parameter Counts (e.g., 8B, 70B, 120b)

- **Meaning:** Indicates the total number of learned parameters (weights and biases) in the

network. "B" stands for Billions.

- **Performance Consideration:** In a dense model (like Llama), a 70B model requires calculating all 70B parameters for every single token generated. At standard 16-bit precision, you need roughly 2GB of VRAM for every 1B parameters just to load the weights (e.g., ~140GB for a 70B model).

Active Parameters in MoE (e.g., A3B, A12B, A35B)

- **Meaning:** "A" denotes **Active Parameters**. This is specific to **Mixture of Experts (MoE)** architectures.
- **Mechanism:** In an MoE model (like Qwen3-Coder-480B-A35B), the feed-forward networks (FCNNs) in the transformer layers are replaced by multiple specialized "expert" sub-networks. A routing algorithm determines which 1 or 2 experts are needed for a specific token.
- **Performance Consideration:** This fundamentally decouples model capacity from inference cost. The model has 480B total parameters of knowledge (requiring massive VRAM to store), but it only activates 35B parameters per token. Compute speed (Tokens/sec) behaves like a 35B model, not a 480B model.

Tuning States (Base vs. Instruct / it)

- **Base:** The foundational model resulting strictly from pre-training on massive corpora via next-token prediction. It does not know how to answer questions; it only knows how to complete documents.
- **Instruct / it:** Instruction-tuned. The base model has undergone Supervised Fine-Tuning (SFT) and alignment (like RLHF or DPO) to understand conversational formats (User/Assistant roles) and respond directly to prompts.

Reasoning Modes (Thinking, R1)

- **Meaning:** Designates models optimized for verifiability and complex logic through **Chain-of-Thought (CoT)** generation.
- **Mechanism:** Trained via large-scale Reinforcement Learning (RL) to emit thousands of "thinking" tokens before emitting the final answer. The model learns to break down problems, correct its own mistakes, and test hypotheses internally.
- **Performance Consideration:** Massive increase in **Time-to-First-Token (TTFT)**. Generating 2,000 hidden thinking tokens before outputting the actual response drastically increases the API cost per query compared to a standard instruct model.

Modality & Specialty Tags (VL, Vision, Coder)

- **VL / Vision:** Vision-Language. Indicates the model architecture has been expanded to include a vision encoder (like SigLIP or CLIP). The encoder processes an image into continuous vector representations, which are then projected directly into the LLM's standard embedding space.

- **Coder:** Indicates continuous pre-training on high-quality code repositories, git commits, and synthetic coding scenarios. These models sacrifice generalized prose ability in exchange for severe exactitude in syntax and tool-calling structures.